

1. Which of the following statements correctly describes a complete binary tree?
a) All levels are completely filled except possibly the last, which is filled from left to right b) Every node has exactly two children c) It must also be a full binary tree d) It always has exactly 2^h nodes at height h
2. What is the worst-case time complexity of inserting a value into an array-based Min-Heap of n elements?
a) $O(1)$ **b) $O(\log n)$** c) $O(n)$ d) $O(n \log n)$
3. Which statement about duplicate values in a heap is correct?
a) Duplicates are never allowed in a heap b) Duplicates are allowed, since the heap property only constrains parent-child ordering c) Duplicates break the heap property d) Duplicates are only allowed at leaf nodes
4. A MaxHeap is stored as the array $[X, 88, 75, 66, 40, 33, 20, 10]$ (index 0 unused). What is the array immediately after one `extractMax()` call?
a) $[X, 75, 66, 20, 40, 33, 10]$ b) $[X, 66, 75, 20, 40, 33, 10]$ c) $[X, 75, 40, 66, 10, 33, 20]$ d) $[X, 10, 75, 66, 40, 33, 20]$
5. A binary tree has In-order: 3, 6, 7, 5, 9, 4, 8 and Pre-order: 5, 6, 3, 7, 4, 9, 8. What is the right child of the root?
a) 9 b) 4 c) 8 d) 3
6. Keys 20, 10, 30, 5, 15, 25, 35, 3 are inserted into an empty BST in that order. What is the height of the resulting tree (a single-node tree has height 0)?
a) 2 b) 3 c) 4 d) 5
7. What is the worst-case time complexity of searching for a value in an array-based Min-Heap of n elements?
a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n \log n)$
8. A perfect binary tree has height 4 (a single node has height 0). How many total nodes does it have?
a) 15 b) 16 c) 31 d) 32
9. For the same perfect binary tree of height 4, how many nodes are in the right subtree of the root?
a) 7 b) 15 c) 16 d) 31
10. During the sink (heapify-down) operation on a Max-Heap, a node's value is compared against:
a) Only its left child b) Only its right child c) The smaller of its two children d) The larger of its two children
11. During the swim (heapify-up) operation on a Min-Heap, a node's value is compared against:
a) Its parent b) Its left child c) Its right child d) Both children
12. A binary tree is stored in a 1-indexed array of size 50 (indices 1..49 hold values). Is the node at index 26 a leaf?
a) Yes b) No c) Cannot be determined d) Only if the tree is complete
13. What is the maximum number of nodes possible in a binary tree of height h (a single node has height 0)?
a) 2^h b) $2^{(h+1)} - 1$ c) $2^h - 1$ d) $h + 1$
14. Which traversal of a Binary Search Tree always visits the keys in ascending sorted order?
a) Pre-order b) In-order c) Post-order d) Level-order
15. Consider the tree: root 50, left child 30 (children 20, 40), right child 70 (children 60, 90). What is its Post-order traversal?
a) 20, 40, 30, 60, 90, 70, 50 b) 50, 30, 20, 40, 70, 60, 90 c) 20, 30, 40, 50, 60, 70, 90 d) 90, 70, 60, 50, 40, 30, 20
16. The Post-order sequence 20, 40, 30, 60, 90, 70, 50 (from the previous question) is inserted, in that order, into an empty Min-Heap. What value ends up at the root?
a) 20 b) 30 c) 50 d) 70

17. A complete binary tree with 100 nodes is stored in a 1-indexed array. What is the index of the parent of the node at index 57?

- a) 27 b) 28 c) 29 d) 57

18. Which of the following arrays (1-indexed, index 0 unused) is NOT a valid Max-Heap?

- a) [X, 90, 70, 80, 40, 30, 60, 20] b) [X, 90, 80, 70, 60, 50, 40, 30] c) [X, 90, 70, 85, 40, 30, 60, 20] d) [X, 50, 90, 30, 20, 10]

19. What is the time complexity of building a heap from an unsorted array of n elements using the bottom-up heapify (Floyd's build-heap) method?

- a) $O(n \log n)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$

20. The keys 1 through 15 are inserted into a BST so that the tree is as balanced as possible (minimum height). What is the height of this tree (a single node has height 0)?

- a) 3 b) 4 c) 7 d) 14

Q1. Signal Decoder (Recursion) [CO4, 8 Points]

A binary tree represents a signal decoder. Each node stores a single bit, 0 or 1. Every root-to-leaf path corresponds to one valid decoded message. A receiver is given a sequence of bits and must check whether following that sequence through the tree successfully decodes a message.

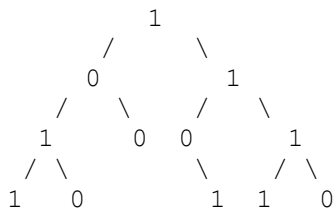
Write a function `decodeSignal()` that takes the root of the binary tree and the sequence of bits (as an int array) to follow. It must return the string "DECODED" if the sequence matches a complete path from the root to a leaf, "PARTIAL" if the sequence matches part of a path but ends at a non-leaf node, and "INVALID" if the sequence cannot be matched to any path in the tree at all.

Assume the Node class is already defined with properties: (int) bit, (Node) left, (Node) right.

Rules:

- You may take only one extra parameter or one helper function.
- You must solve this using recursion only.
- No extra data structures (array/linked list) or static/global variables are allowed.

Sample tree (root at top):



Examples:

```
bits = {1,0,1,0} -> DECODED    (a full root-to-leaf path)
bits = {1,1,0}   -> PARTIAL    (matches part of a path, stops at a non-leaf)
bits = {0,1,1}   -> INVALID    (no such path exists in the tree)
```

Python Notation:

```
def decodeSignal(root, bits):
    # TODO
```

Java Notation:

```
public static String decodeSignal(Node root, int[] bits) {
    // TODO
}
```

Q2. BST Construction and Balance Check [CO3, 6 Points]

Insert the keys 40, 20, 60, 10, 30, 50, 70, 5 into an empty Binary Search Tree, one at a time, in the order given.

- Draw the resulting BST. (3 marks)
- State the height of the tree (a single-node tree has height 0). (1 mark)
- A tree is height-balanced if, for every node, the heights of its left and right subtrees differ by at most 1. Determine whether the resulting tree is height-balanced at every node. Show the balance factor for at least three internal nodes to justify your answer. (2 marks)

Q3. Min-Heap extractMin() Simulation [CO1, 4 Points]

A Min-Heap is represented as the array [X, 5, 12, 8, 25, 30, 15, 10] (1-indexed, index 0 unused).

Simulate two consecutive extractMin() calls. For each call, show:

- the value removed,
- the array immediately after moving the last element to the root,
- every swap performed while sinking, and
- the final array after the sink completes.

Q4. Print Job Scheduler (Heap-based Greedy) [CO1, 5 Points]

A printer receives an array of jobs, where each element represents the priority of that job, and is given a capacity (number of jobs it can print before its queue closes for the day). Your task is to print the possible jobs within that capacity, following these rules:

- Each printed job takes 1 print slot.
- At each step, print the job with the highest priority remaining.
- Ignore any jobs left over once capacity is reached.

The time complexity of your solution must be less than $O(n^2)$. Assume a MaxHeap class is already provided with insert(value) and extract(), where extract() removes and returns the maximum element.

Caution: a nested loop / repeated linear scan for the maximum will result in $O(n^2)$ time complexity.

Example:

```
jobs = [12, 55, 40, 18, 33], capacity = 3
Slot 1 - Priority 55
Slot 2 - Priority 40
Slot 3 - Priority 33
Jobs printed: 3, Jobs ignored: 2
```

Python Notation:

```
def printJobs(jobs, capacity):
    # TODO
```

Java Notation:

```
static void printJobs(int[] jobs, int capacity) {
    // TODO
}
```

Q5. Signal Booster (Recursive Tree Modification) [CO4, 8 Points]

A sensor network is modeled as a binary tree of integer readings. A booster process starts at the root and propagates downward, but it can only affect nodes with an EVEN value.

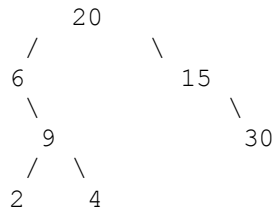
- If an even-valued node has both a left and a right child, the booster cannot act on that node; it simply continues to both children.
- If an even-valued node has exactly one child, it absorbs the entire subtree rooted at that child: the sum of all values in that subtree is added to the node's own value, and every node in that subtree is removed.
- If an even-valued node is a leaf, the booster has no effect on it.
- Odd-valued nodes are never affected, regardless of how many children they have.

Complete the function `signalBooster()` to process the tree according to these rules and return the root of the modified tree.

Assume the `Node` class is already defined with `elem (int)`, `left (Node)`, `right (Node)`.

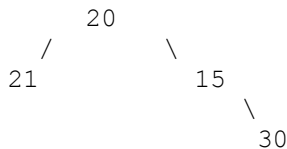
You cannot use any static/instance variables or any extra data structures. You are not allowed to create any new `Node` objects. You are allowed to use helper methods.

Sample given tree:



Explanation: node 6 is even with a single child (9); the subtree rooted at 9 sums to $9+2+4=15$, so 6 becomes $6+15=21$ and its subtree is removed. Node 15 is odd, so it is left untouched even though it has one child. Node 20 is even but has two children, so it is only passed through.

Returned tree:



Python Notation:

```
def signalBooster(root):
    # TODO
```

Java Notation:

```
static Node signalBooster(Node root) {
    // TODO
}
```

Q6. Searching a Product Catalogue Stored as a BST Array [CO4, 4 Points]

A product catalogue is stored as a complete Binary Search Tree in a 1-indexed array of n elements. Write `searchCatalogArr()` which searches for a target ID using BST properties, and prints the sequence of values it checks along the way. If the target is not found, still print the checked sequence, ending it with "Not_Found".

Note: if a node is at index i , its left child is at $2i$ and its right child is at $2i+1$. Your solution must run in $O(\log n)$ time and must not use any other data structure.

Sample array: `arr = [X, 50, 25, 75, 10, 30, 60, 90]`
target = 60 -> Output: 50, 75, 60
target = 45 -> Output: 50, 25, 30, Not_Found

Python Notation:

```
def searchCatalogArr(arr, target):
    # TODO
```

Java Notation:

```
static void searchCatalogArr(int[] arr, int target) {
    // TODO
}
```

Q7. Odd-Only Filter on a Min-Heap [CO1, 4 Points]

Write a function `oddOnly()` that takes a `MinHeap` object as a parameter and returns a `MinHeap` object containing only the odd-valued elements from it. The function should also print the number of even values that were discarded.

Assume a `MinHeap` class is already implemented with public methods `insert()`, `extractMin()`, and `getSize()`. Only public methods may be used. You may create extra `MinHeap` objects to solve this, but you must use `MinHeap` operations only, not any other data structure.

Given Heap Object: `MinHeap: [X, 4, 7, 10, 15, 8, 20, 13]`

Sample Output: Discarded: 4

Explanation: 4, 10, 8, 20 are even and discarded; 7, 15, 13 are odd and kept, giving the returned heap `[X, 7, 15, 13]`.

Python Notation:

```
def oddOnly(heap):  
    # TODO
```

Java Notation:

```
static MinHeap oddOnly(MinHeap heap) {  
    // TODO  
}
```

Q8. Level-Parity Sums on a Perfect Binary Tree [CO4, 6 Points]

You are given the root of a perfect binary tree with 15 nodes, numbered 1 through 15 in breadth-first order (the root is 1, its children are 2 and 3, and so on down to the leaves 8-15). The root is at level 0.

Write `levelParitySum(root)` that prints two sums: the total of all node values located at even levels (0, 2, 4, ...) and the total of all node values located at odd levels (1, 3, 5, ...).

You cannot use any extra data structures beyond simple counters/accumulators; recursion or a manual level-tracking traversal is expected.

For this specific tree, level 0 = {1}, level 1 = {2,3}, level 2 = {4,5,6,7}, level 3 = {8,...,15}.

Expected output: Even levels: 23, Odd levels: 97

Python Notation:

```
def levelParitySum(root):  
    # TODO
```

Java Notation:

```
static void levelParitySum(Node root) {  
    // TODO  
}
```

Q9. Reconstructing Post-order from Pre-order and In-order [CO3, 5 Points]

A binary tree has the following traversals:

Pre-order: 8, 5, 3, 7, 9, 15, 12, 20

In-order: 3, 5, 7, 8, 9, 12, 15, 20

Reconstruct the tree (you may draw it or describe it clearly) and write down its Post-order traversal. Show the key reasoning steps you used to identify each subtree's root.

Q10. k Largest Elements using a Bounded Min-Heap [CO2, 6 Points]

Write `findKLargest(numbers, k)` that returns the k largest elements of an integer array, in ascending order, using a Min-Heap of size at most k so that the overall algorithm runs in $O(n \log k)$ time rather than $O(n \log n)$.

Hint: keep a Min-Heap that never grows past k elements. For each new number, if the heap has fewer than k elements insert it directly; otherwise compare the number with the heap's minimum and replace the minimum only if the new number is larger.

Assume a `MinHeap` class is already provided with `insert()`, `extractMin()`, `peekMin()`, and `getSize()`.

Example:

```
numbers = [9, 3, 15, 2, 20, 8, 1], k = 3
findKLargest(numbers, k) -> [9, 15, 20]
```

Python Notation:

```
def findKLargest(numbers, k):
    # TODO
```

Java Notation:

```
static int[] findKLargest(int[] numbers, int k) {
    // TODO
}
```